

LAB SHEET

06026212 DATA WAREHOUSING

สัปดาห์ที่ 7: การออกแบบ Fact Table

Fact Table Design ด้วย dbt

ระยะเวลา 2 ชั่วโมง | PostgreSQL + dbt + Docker | orders_log.csv (2,000 แถว)

แนวคิดหลัก: ออกแบบและสร้าง Transaction Fact, Periodic Snapshot และ Accumulating Snapshot ด้วย dbt โดยกำหนด Grain, Measures, Keys และ Tests ให้สอดคล้องกับ KPI/SLA

วัตถุประสงค์

เมื่อสิ้นสุด Lab นี้ นักศึกษาจะสามารถ:

- เลือกประเภท Fact Table (Transaction / Periodic Snapshot / Accumulating Snapshot / Factless Fact) ให้เหมาะสมกับ KPI
- จำแนก Measures เป็น Additive, Semi-additive และ Non-additive ได้
- ออกแบบ Fact Table ที่มี Foreign Keys, Degenerate Dimension และ Audit Columns
- สร้าง ทดสอบ และตรวจสอบ Fact Table ใน PostgreSQL ผ่านโมเดล dbt

เครื่องมือและชุดข้อมูล

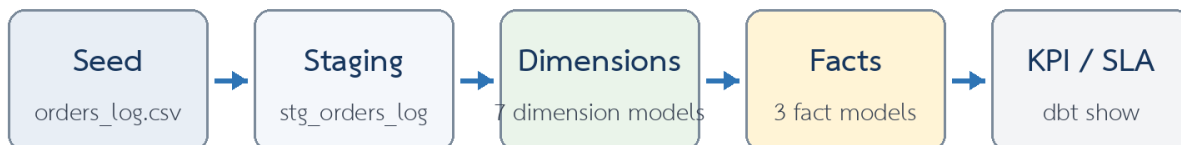
เครื่องมือ	ใช้ทำอะไร
PostgreSQL	ฐานข้อมูลหลักที่ทำงานผ่าน Docker
dbt	สร้างโมเดล จัดลำดับ dependency โหลด seed และทดสอบคุณภาพข้อมูล
pgAdmin	ตรวจสอบ schema ตาราง และผลลัพธ์ในฐานข้อมูล
Metabase	ทางเลือกสำหรับทดลองสร้างคำถามหรือ Dashboard จาก Fact Table
Docker	สร้างสภาพแวดล้อมจำลองจาก DWH_Lab(3).zip

- ไฟล์: orders_log.csv
- จำนวนข้อมูล: 2,000 แถว และ order_id ไม่ซ้ำกัน 2,000 ค่า
- ช่วงวันที่: 2024-01-01 ถึง 2025-07-05 เมื่อรวม order/shipped/delivered date
- สถานะ: ORDERED, SHIPPED, DELIVERED และ CANCELLED

กำหนด Grain ก่อนเริ่มสร้าง Fact

ในชุดข้อมูลปัจจุบัน 1 แถวแทน 1 รายการใน orders_log และ order_id ไม่ซ้ำกัน จึงสามารถใช้ order_id เป็น Degenerate Dimension และทดสอบ unique ได้ หากระบบจริงมีหลายสินค้าในหนึ่ง order ต้องเพิ่ม order_line_id หรือคีย์ระดับบรรทัดก่อนสร้าง Transaction Fact

สายการแปลงข้อมูลใน Lab 7



ภาพรวม lineage ของ Lab 7

ส่วนที่ 1 จำแนก Measures ตาม Additivity

ให้นักศึกษาจำแนก Measures ใน Google Form โดยพิจารณาจาก Grain และมีติที่นำไปรวม
ไม่พิจารณาจากชื่อคอลัมน์เพียงอย่างเดียว

ส่วนที่ 2 เตรียมสภาพแวดล้อมและ dbt project

2.1 เปิด services และสร้างฐานข้อมูล

```
CREATE DATABASE lab7;
```

2.2 จัดเตรียมโฟลเดอร์

สร้างไฟล์และโฟลเดอร์ต่อไปนี้ภายใน DWH_Lab

```
dbt_root/
├── profiles.yml
└── dbt/
    ├── lab7/
    │   ├── dbt_project.yml
    │   ├── seeds/
    │   │   ├── orders_log.csv
    │   │   └── properties.yml
    │   ├── models/
    │   │   ├── staging/
    │   │   │   └── stg_orders_log.sql
    │   │   ├── dimensions/
    │   │   │   ├── dim_date.sql
    │   │   │   ├── dim_product.sql
    │   │   │   ├── dim_customer.sql
    │   │   │   ├── dim_store.sql
    │   │   │   ├── dim_staff.sql
    │   │   │   ├── dim_payment_method.sql
    │   │   │   └── dim_order_status.sql
    │   │   ├── facts/
    │   │   │   ├── fct_orders_txn.sql
    │   │   │   ├── fct_orders_daily_snapshot.sql
    │   │   │   └── fct_orders_lifecycle.sql
    │   │   └── schema.yml
    │   └── tests/
    │       └── assert_daily_snapshot_grain.sql
```

ตำแหน่งไฟล์ dataset

คัดลอก orders_log.csv ไปไว้ที่ dbt/lab7/seeds/orders_log.csv

2.3 กำหนดการเชื่อมต่อ PostgreSQL

dbt_root/profiles.yml

```
lab7:
  target: dev
  outputs:
    dev:
      type: postgres
      host: postgres
      port: 5432
      user: dw_user
      password: dw_pass
      dbname: lab7
      schema: dbt
      threads: 4
```

2.4 กำหนดค่า Project dbt

dbt/lab7/dbt_project.yml

```

name: 'lab7'
version: '1.0.0'
config-version: 2

profile: 'lab7'

model-paths: ['models']
seed-paths: ['seeds']
test-paths: ['tests']
macro-paths: ['macros']

clean-targets:
  - 'target'
  - 'dbt_packages'

models:
  staging:
    +materialized: view
  dimensions:
    +materialized: table
  facts:
    +materialized: table

```

กำหนด seed เป็น varchar ก่อน เพื่อให้ staging เป็นจุดเดียวที่ควบคุมการ cast ชนิดข้อมูล และรักษารหัสที่อาจมีเลขศูนย์นำหน้า

2.5 ตรวจสอบ Seed

Dbt/lab7/seeds/properties.yml

```

version: 2

seeds:
  - name: orders_log
    description: 'ข้อมูลคำสั่งซื้อดิบจาก orders_log.csv'
    config:
      column_types:
        order_id: varchar(20)
        order_date: varchar(10)
        shipped_date: varchar(10)
        delivered_date: varchar(10)
        product_code: varchar(10)
        product_name: varchar(100)
        category: varchar(50)
        size: varchar(10)
        unit_price: varchar(30)
        quantity: varchar(30)
        discount_amount: varchar(30)
        net_amount: varchar(30)
        cost_amount: varchar(30)
        margin_amount: varchar(30)
        customer_code: varchar(10)
        customer_name: varchar(100)
        store_code: varchar(10)
        store_name: varchar(100)
        staff_code: varchar(10)
        staff_name: varchar(100)
        payment_method: varchar(50)
        status: varchar(20)
      columns:
        - name: order_id
          tests:
            - not_null
            - unique
        - name: status
          tests:
            - accepted_values:
                values: ['ORDERED', 'SHIPPED', 'DELIVERED', 'CANCELLED']

```

2.6 เริ่ม Docker และโหลด Seed

เปิด Terminal ที่โฟลเดอร์ DWH_Lab แล้วรันคำสั่งตามลำดับ

```

docker exec -it dw_dbt bash
cd lab7
dbt debug
dbt seed --full-refresh
dbt show --select orders_log

```

จุดตรวจสอบ	ค่าที่ควรได้
dbt debug	All checks passed
dbt seed	โหลด orders_log สำเร็จ 2,000 แถว
ตำแหน่งใน PostgreSQL	database lab7, schema dbt, table orders_log

ส่วนที่ 3 สร้าง Staging และ Dimension Models

3.1 Staging Model

Staging model ทำหน้าที่ trim ข้อความ แปลงค่าว่างเป็น NULL และ cast ชนิดข้อมูลให้พร้อมใช้ โดยยังรักษา grain เดิม 1 แถวต่อรายการ

[dbt/lab7/models/staging/stg_orders_log.sql](#)

```
{{ config(materialized='view') }}

select
  trim(order_id) as order_id,
  nullif(trim(order_date), '')::date as order_date,
  nullif(trim(shipped_date), '')::date as shipped_date,
  nullif(trim(delivered_date), '')::date as delivered_date,
  trim(product_code) as product_code,
  trim(product_name) as product_name,
  trim(category) as category,
  nullif(trim(size), '') as size,
  nullif(trim(unit_price), '')::numeric(12, 2) as unit_price,
  nullif(trim(quantity), '')::integer as quantity,
  nullif(trim(discount_amount), '')::numeric(12, 2)
    as discount_amount,
  nullif(trim(net_amount), '')::numeric(12, 2) as net_amount,
  nullif(trim(cost_amount), '')::numeric(12, 2) as cost_amount,
  nullif(trim(margin_amount), '')::numeric(12, 2) as margin_amount,
  trim(customer_code) as customer_code,
  trim(customer_name) as customer_name,
  trim(store_code) as store_code,
  trim(store_name) as store_name,
  trim(staff_code) as staff_code,
  trim(staff_name) as staff_name,
  trim(payment_method) as payment_method,
  trim(status) as status
from {{ ref('orders_log') }}
```

3.2 Date Dimension

สร้างวันที่ต่อเนื่องตั้งแต่วันสั่งซื้อแรกถึง milestone ล่าสุด เพื่อให้ Periodic Snapshot มีวันที่ที่ไม่มีรายการได้

[dbt/lab7/models/dimensions/dim_date.sql](#)

```
with bounds as (
  select
    min(order_date) as min_date,
    max(
      greatest(
        order_date,
        coalesce(shipped_date, order_date),
        coalesce(delivered_date, order_date)
      )
    ) as max_date
  from {{ ref('stg_orders_log') }}
),

date_spine as (
  select g.date_day::date as full_date
  from bounds
  cross join lateral generate_series(
    bounds.min_date,
    bounds.max_date,
    interval '1 day'
  ) as g(date_day)
)

select
  to_char(full_date, 'YYYYMMDD')::integer as date_key,
  full_date,
  extract(year from full_date)::integer as year,
  extract(month from full_date)::integer as month,
  extract(day from full_date)::integer as day,
```

```
(
  extract(year from full_date)::integer * 100
  + extract(month from full_date)::integer
) as yyyyymm
from date_spine
```

3.3 Product Dimension

[dbt/lab7/models/dimensions/dim_product.sql](#)

```
select
  md5(product_code) as product_key,
  product_code,
  max(product_name) as product_name,
  max(category) as category,
  case
    when bool_or(size in ('S', 'M', 'L')) then 'S/M/L'
    else '-'
  end as size_domain
from {{ ref('stg_orders_log') }}
group by product_code
```

3.4 Customer, Store และ Staff Dimensions

[dbt/lab7/models/dimensions/dim_customer.sql](#)

```
select
  md5(customer_code) as customer_key,
  customer_code,
  max(customer_name) as customer_name
from {{ ref('stg_orders_log') }}
group by customer_code
```

[dbt/lab7/models/dimensions/dim_store.sql](#)

```
select
  md5(store_code) as store_key,
  store_code,
  max(store_name) as store_name
from {{ ref('stg_orders_log') }}
group by store_code
```

[dbt/lab7/models/dimensions/dim_staff.sql](#)

```
select
  md5(staff_code) as staff_key,
  staff_code,
  max(staff_name) as staff_name
from {{ ref('stg_orders_log') }}
group by staff_code
```

3.5 Payment Method และ Order Status Dimensions

[dbt/lab7/models/dimensions/dim_payment_method.sql](#)

```
select
  md5(payment_method) as payment_method_key,
  payment_method
from {{ ref('stg_orders_log') }}
group by payment_method
```

[dbt/lab7/models/dimensions/dim_order_status.sql](#)

```
select
  md5(status) as status_key,
  status
from {{ ref('stg_orders_log') }}
group by status
```

ส่วนที่ 4 สร้าง Fact Models

4.1 Transaction Fact

- Grain: 1 แถว = 1 รายการใน orders_log (ข้อมูลชุดนี้มี 1 แถวต่อ order_id)

- Degenerate Dimension: order_id
- Additive Measures: quantity, gross_amount, discount_amount, net_amount, cost_amount, margin_amount
- Audit Columns: load_datetime, batch_id, source_system

dbt/lab7/models/facts/fact_orders_txn.sql

```
select
  md5(s.order_id) as order_fact_key,

  -- Degenerate dimension
  s.order_id,

  -- Foreign keys
  d.date_key as order_date_key,
  p.product_key,
  c.customer_key,
  st.store_key,
  sf.staff_key,
  pm.payment_method_key,
  os.status_key,

  -- Measures
  s.unit_price,
  s.quantity,
  (s.unit_price * s.quantity)::numeric(14, 2) as gross_amount,
  s.discount_amount,
  s.net_amount,
  s.cost_amount,
  s.margin_amount,

  -- Audit columns
  cast('{{ run_started_at }}' as timestamptz) as load_datetime,
  '{{ invocation_id }}' as batch_id,
  'orders_log.csv'::varchar as source_system
from {{ ref('stg_orders_log') }} as s
join {{ ref('dim_date') }} as d
  on s.order_date = d.full_date
join {{ ref('dim_product') }} as p
  on s.product_code = p.product_code
join {{ ref('dim_customer') }} as c
  on s.customer_code = c.customer_code
join {{ ref('dim_store') }} as st
  on s.store_code = st.store_code
join {{ ref('dim_staff') }} as sf
  on s.staff_code = sf.staff_code
join {{ ref('dim_payment_method') }} as pm
  on s.payment_method = pm.payment_method
join {{ ref('dim_order_status') }} as os
  on s.status = os.status
```

4.2 Periodic Snapshot Fact

- Grain: 1 แถว = 1 วัน × 1 สาขา × 1 สินค้า
- สร้าง grid ครบทุกวัน-สาขา-สินค้า และเติม 0 เมื่อไม่มีรายการ
- Snapshot Measures ในชุดนี้เป็น daily flow จึงรวมข้ามเวลาได้

dbt/lab7/models/facts/fact_orders_daily_snapshot.sql

```

with snapshot_grid as (
  select
    d.date_key as snapshot_date_key,
    s.store_key,
    p.product_key
  from {{ ref('dim_date') }} as d
  cross join {{ ref('dim_store') }} as s
  cross join {{ ref('dim_product') }} as p
),

daily_activity as (
  select
    order_date_key as snapshot_date_key,
    store_key,
    product_key,
    count(distinct order_id)::integer as orders_count,
    sum(quantity)::integer as qty_sold,
    sum(gross_amount)::numeric(14, 2) as gross_amount,
    sum(discount_amount)::numeric(14, 2) as discount_amount,
    sum(net_amount)::numeric(14, 2) as net_amount,
    sum(margin_amount)::numeric(14, 2) as margin_amount
  from {{ ref('fact_orders_txn') }}
  group by order_date_key, store_key, product_key
)

select
  g.snapshot_date_key,
  g.store_key,
  g.product_key,
  coalesce(a.orders_count, 0) as orders_count,
  coalesce(a.qty_sold, 0) as qty_sold,
  coalesce(a.gross_amount, 0)::numeric(14, 2) as gross_amount,
  coalesce(a.discount_amount, 0)::numeric(14, 2)
    as discount_amount,
  coalesce(a.net_amount, 0)::numeric(14, 2) as net_amount,
  coalesce(a.margin_amount, 0)::numeric(14, 2) as margin_amount,
  cast('{{ run_started_at }}' as timestamp) as load_datetime,
  '{{ invocation_id }}' as batch_id
from snapshot_grid as g
left join daily_activity as a
  on g.snapshot_date_key = a.snapshot_date_key
  and g.store_key = a.store_key
  and g.product_key = a.product_key

```

เหตุผลที่ไม่ใช้ GROUP BY อย่างเดียว

ถ้า aggregate เฉพาะวันที่มีรายการ ผลลัพธ์จะเป็น Aggregate Fact มากกว่า Periodic Snapshot แบบเต็ม การสร้าง grid ทำให้ทุก period มีแถว แม้ measures เป็น 0 ซึ่งเหมาะกับการวิเคราะห์วันที่ไม่มีการขายและ Dashboard ที่ต้องการแกนเวลาต่อเนื่อง

4.3 Accumulating Snapshot Fact

- Grain: 1 แถว = 1 คำสั่งซื้อ
- Role-playing Date Keys: order_date_key, shipped_date_key, delivered_date_key
- Non-additive Measures: days_to_ship, days_to_deliver, days_ship_to_deliver
- Materialization: incremental โดยใช้ order_id เป็น unique_key

dbt/lab7/models/facts/fact_orders_lifecycle.sql

```

{{
  config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='delete+insert'
  )
}}

select
  s.order_id,
  od.date_key as order_date_key,
  sd.date_key as shipped_date_key,
  dd.date_key as delivered_date_key,

  case
    when s.shipped_date is not null
      then s.shipped_date - s.order_date
  end as days_to_ship,

  case
    when s.delivered_date is not null
      then s.delivered_date - s.order_date

```

```

end as days_to_deliver,
case
  when s.shipped_date is not null
    and s.delivered_date is not null
    then s.delivered_date - s.shipped_date
end as days_ship_to_deliver,

os.status_key as current_status_key,
cast('{{ run_started_at }}' as timestampz) as load_datetime,
('{{ invocation_id }}' as batch_id
from {{ ref('stg_orders_log') }} as s
left join {{ ref('dim_date') }} as od
  on s.order_date = od.full_date
left join {{ ref('dim_date') }} as sd
  on s.shipped_date = sd.full_date
left join {{ ref('dim_date') }} as dd
  on s.delivered_date = dd.full_date
join {{ ref('dim_order_status') }} as os
  on s.status = os.status

```

พฤติกรรม Incremental

เมื่อข้อมูล order เดิมมี shipped_date, delivered_date หรือ status เปลี่ยน dbt จะใช้ order_id หาแถวเดิม แล้วแทนที่ด้วยสถานะล่าสุด เนื่องจาก source ไม่มี updated_at ตัวอย่างนี้ยังอ่าน source ทั้งหมดในแต่ละรอบ แต่แสดงหลักการ update ของ Accumulating Snapshot

ส่วนที่ 5 กำหนด Data Tests และ Build

5.1 Tests ของ Models

dbt model ไม่ได้สร้าง Foreign Key constraint แบบ CREATE TABLE เดิม จึงใช้ relationships test ตรวจสอบ Referential Integrity และใช้ unique/not_null ยืนยัน grain

[dbt/lab7/models/schema.yml](#)

```

version: 2

models:
  - name: stg_orders_log
    description: 'ข้อมูลคำสั่งซื้อที่ cast ชนิดข้อมูลแล้ว'
    columns:
      - name: order_id
        tests: [not_null, unique]
      - name: order_date
        tests: [not_null]
      - name: product_code
        tests: [not_null]
      - name: customer_code
        tests: [not_null]
      - name: store_code
        tests: [not_null]
      - name: staff_code
        tests: [not_null]

  - name: dim_date
    columns:
      - name: date_key
        tests: [not_null, unique]
      - name: full_date
        tests: [not_null, unique]

  - name: dim_product
    columns:
      - name: product_key
        tests: [not_null, unique]
      - name: product_code
        tests: [not_null, unique]

  - name: dim_customer
    columns:
      - name: customer_key
        tests: [not_null, unique]
      - name: customer_code
        tests: [not_null, unique]

  - name: dim_store
    columns:
      - name: store_key
        tests: [not_null, unique]

```



```

- name: store_code
  tests: [not_null, unique]

- name: dim_staff
  columns:
  - name: staff_key
    tests: [not_null, unique]
  - name: staff_code
    tests: [not_null, unique]

- name: dim_payment_method
  columns:
  - name: payment_method_key
    tests: [not_null, unique]
  - name: payment_method
    tests: [not_null, unique]

- name: dim_order_status
  columns:
  - name: status_key
    tests: [not_null, unique]
  - name: status
    tests: [not_null, unique]

- name: fact_orders_txn
  description: "Transaction Fact ที่ grain ระดับรายการใน orders_log"
  columns:
  - name: order_fact_key
    tests: [not_null, unique]
  - name: order_id
    tests: [not_null, unique]
  - name: order_date_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_date')
          field: date_key
  - name: product_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_product')
          field: product_key
  - name: customer_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_customer')
          field: customer_key
  - name: store_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_store')
          field: store_key
  - name: staff_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_staff')
          field: staff_key
  - name: payment_method_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_payment_method')
          field: payment_method_key
  - name: status_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_order_status')
          field: status_key

- name: fact_orders_daily_snapshot
  description: "Periodic Snapshot ระดับวัน-สาขา-สินค้า"
  columns:
  - name: snapshot_date_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_date')
          field: date_key
  - name: store_key
    tests:
      - not_null
      - relationships:
          to: ref('dim_store')
          field: store_key

```

```

- name: product_key
  tests:
    - not_null
  relationships:
    to: ref('dim_product')
    field: product_key

- name: fact_orders_lifecycle
  description: 'Accumulating Snapshot ระดับคำสั่งซื้อ'
  columns:
    - name: order_id
      tests: [not_null, unique]
    - name: order_date_key
      tests:
        - not_null
        - relationships:
            to: ref('dim_date')
            field: date_key
    - name: shipped_date_key
      tests:
        - relationships:
            to: ref('dim_date')
            field: date_key
    - name: delivered_date_key
      tests:
        - relationships:
            to: ref('dim_date')
            field: date_key
    - name: current_status_key
      tests:
        - not_null
        - relationships:
            to: ref('dim_order_status')
            field: status_key

```

5.2 Singular Test สำหรับ Grain ของ Daily Snapshot

[dbt/lab7/tests/assert_daily_snapshot_grain.sql](#)

```

select
  snapshot_date_key,
  store_key,
  product_key,
  count(*) as row_count
from {{ ref('fact_orders_daily_snapshot') }}
group by snapshot_date_key, store_key, product_key
having count(*) > 1

```

5.3 Build Project

```

dbt build

dbt ls --resource-type model

dbt show --inline "select count(*) as row_count from {{ ref('fact_orders_txn') }}"

dbt show --inline "select count(*) as row_count from {{ ref('fact_orders_daily_snapshot') }}"

dbt show --inline "select count(*) as row_count from {{ ref('fact_orders_lifecycle') }}"

```

Model	จำนวนแถวที่โหลดหึ่ง	ที่มา
stg_orders_log	2,000	เท่ากับ seed
dim_date	552	วันที่ต่อเนื่อง 2024-01-01 ถึง 2025-07-05
dim_product	12	product_code ไม่ซ้ำ
dim_customer	484	customer_code ที่ปรากฏ
dim_store	6	store_code ไม่ซ้ำ
dim_staff	6	staff_code ไม่ซ้ำ
dim_payment_method	4	วิธีชำระเงิน
dim_order_status	4	สถานะคำสั่งซื้อ
fact_orders_txn	2,000	1 แถวต่อ order_id
fact_orders_daily_snapshot	39,744	552 วัน × 6 สาขา × 12 สินค้า
fact_orders_lifecycle	2,000	1 แถวต่อ order_id

ส่วนที่ 6 ทดลอง Query ตาม KPI/SLA ด้วย dbt

ใช้ dbt show --inline เพื่อ compile Jinja, รัน SQL กับ PostgreSQL และแสดงผลใน Terminal โดยไม่สร้างตาราง KPI เพิ่ม

6.1 ยอดขายรวมต่อวันจาก Transaction Fact

```
dbt show --inline "
select
  d.full_date as order_date,
  sum(t.net_amount) as total_revenue
from {{ ref('fact_orders_txn') }} as t
join {{ ref('dim_date') }} as d
  on t.order_date_key = d.date_key
group by d.full_date
order by d.full_date
" --limit 20
```

6.2 ยอดขายรวมต่อวันจาก Daily Snapshot

```
dbt show --inline "
select
  d.full_date as snapshot_date,
  sum(s.net_amount) as total_revenue
from {{ ref('fact_orders_daily_snapshot') }} as s
join {{ ref('dim_date') }} as d
  on s.snapshot_date_key = d.date_key
group by d.full_date
order by d.full_date
" --limit 20
```

การเทียบผลสอง Fact

วันที่มีรายการ ยอดรวมจาก Transaction Fact และ Daily Snapshot ต้องเท่ากัน ส่วน Daily Snapshot จะแสดงวันที่ไม่มีรายการเป็น 0 ด้วย เพราะมีแถวครบทุกวัน

6.3 Top 5 สินค้าที่ทำกำไรมากที่สุด

```
dbt show --inline "
select
  p.product_code,
  p.product_name,
  sum(t.margin_amount) as total_margin
from {{ ref('fact_orders_txn') }} as t
join {{ ref('dim_product') }} as p
  on t.product_key = p.product_key
group by p.product_code, p.product_name
order by total_margin desc
" --limit 5
```

ลำดับ	สินค้า	total_margin ที่คาดหวัง
1	P004 Cappuccino	5,850.30
2	P003 Latte	5,539.52
3	P202 Cheesecake	5,138.94
4	P005 Mocha	5,074.15
5	P001 Espresso	4,863.33

6.4 SLA เฉลี่ย: วันจากสั่งซื้อถึงส่งสำเร็จ

```
dbt show --inline "
select
  avg(days_to_ship)::numeric(10, 4) as avg_days_to_ship,
```

```

avg(days_to_deliver)::numeric(10, 4) as avg_days_to_deliver,
avg(days_ship_to_deliver)::numeric(10, 4)
  as avg_days_ship_to_deliver
from {{ ref('fact_orders_lifecycle') }}
where days_to_deliver is not null
"

```

Measure	ค่าที่คาดหวังโดยประมาณ
avg_days_to_ship	0.9870 วัน
avg_days_to_deliver	2.9800 วัน
avg_days_ship_to_deliver	1.9913 วัน

6.5 ยอดขายแยกตามวิธีชำระเงิน

```

dbt show --inline "
select
  pm.payment_method,
  sum(t.net_amount) as revenue
from {{ ref('fact_orders_txn') }} as t
join {{ ref('dim_payment_method') }} as pm
  on t.payment_method_key = pm.payment_method_key
group by pm.payment_method
order by revenue desc
"

```

payment_method	revenue ที่คาดหวัง
QR PromptPay	49,042.75
Mobile Wallet	47,088.88
Cash	45,047.18
Credit Card	42,734.80

6.6 อัตรายกเลิกออเดอร์

```

dbt show --inline "
select
  100.0
  * sum(case when os.status = 'CANCELLED' then 1 else 0 end)
  / count(*) as cancel_rate_pct
from {{ ref('fact_orders_txn') }} as t
join {{ ref('dim_order_status') }} as os
  on t.status_key = os.status_key
"

```

ผลลัพธ์ตรวจสอบ

ข้อมูลมี CANCELLED 173 รายการจาก 2,000 รายการ ดังนั้น cancel_rate_pct เท่ากับ 8.65% measure นี้เป็น Non-additive และต้องคำนวณใหม่จากจำนวนยกเลิกหารด้วยจำนวนทั้งหมด

ส่วนที่ 7 สรุปแนวคิด Fact Table

Fact Type	Grain ใน Lab	เหมาะกับคำถาม
Transaction Fact	1 รายการใน orders_log	ยอดขาย ค่าไร จำนวนสินค้า แยกตามมิติต่าง ๆ
Periodic Snapshot	1 วัน × 1 สาขา × 1 สินค้า	แนวโน้มรายวัน วันที่ไม่มีรายการ และ Dashboard
Accumulating Snapshot	1 คำสั่งซื้อ	ระยะเวลาแต่ละ milestone และ SLA ของกระบวนการ
Factless Fact	ขึ้นกับ event/coverage	นับเหตุการณ์ที่ไม่มี numeric measure เช่น

Fact Type	Grain ใน Lab	เหมาะกับคำถาม
		การเข้าร่วมกิจกรรม

ส่วนที่ 8 สิ่งที่ต้องส่ง

รายการ	รูปแบบ
จำแนก Measures ตาม Additivity	ตอบใน Google Form
ผลลัพธ์ Query ตาม KPI/SLA	กรอกค่าผลลัพธ์ใน Google Form